

Predykcja cen samochodów z wykorzystaniem algorytmów uczenia maszynowego

Autorzy: Oleksandr Babenkov & Piotr Bajerowski

7 czerwca 2026

Spis treści

1	Cel projektu	3
2	Wybór i pobranie zbioru danych	3
3	Charakterystyka pierwszego zbioru danych	4
3.1	Pierwsze pięć rekordów	4
3.2	Struktura danych	4
3.3	Statystyki opisowe	5
4	Wstępna analiza i wizualizacja danych	5
4.1	Korelacje z ceną	6
5	Podział danych	9
6	Przygotowanie danych do uczenia maszynowego	9
7	Modele bazowe	10
8	RandomForestRegressor	11
9	SVR	11
10	MLPRegressor	12
11	Porównanie wyników dla pierwszego zbioru danych	13
12	Analiza wyników innych notebooków Kaggle	13
13	Drugi eksperyment na innym zbiorze danych	13
14	Charakterystyka zmiennych w drugim zbiorze danych	14
15	Podział drugiego zbioru danych	14
16	Preprocessing drugiego zbioru danych	15
17	Wyniki modeli na drugim zbiorze danych	15

18 Krzywe uczenia	16
19 Ważność cech w modelu Random Forest	19
20 Ewaluacja końcowa na zbiorze testowym	20
21 Porównanie działania algorytmów	20
22 Wnioski końcowe	21

1 Cel projektu

Celem projektu było opracowanie i porównanie modeli uczenia maszynowego służących do predykcji cen samochodów na podstawie ich parametrów technicznych oraz eksploatacyjnych. W projekcie wykorzystano algorytmy regresyjne dostępne w pakiecie `scikit-learn`: `RandomForestRegressor`, `SVR` oraz `MLPRegressor`. Dodatkowo wykorzystano biblioteki `NumPy`, `pandas` oraz `matplotlib`, a także narzędzia wspierające preprocessing danych.

Jako podstawową metrykę jakości w pierwszym etapie projektu przyjęto RMSE, czyli pierwiastek błędu średniokwadratowego:

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} \quad (1)$$

gdzie y_i oznacza rzeczywistą cenę samochodu, a $\hat{y}_i$ cenę przewidzianą przez model. RMSE wyrażone jest w tej samej jednostce co zmienna docelowa, czyli w dolarach. Metryka ta silniej karze duże błędy predykcji.

2 Wybór i pobranie zbioru danych

W pierwotnym poleceniu projektowym wskazany został zbiór danych dostępny pod adresem:

```
https://raw.githubusercontent.com/murpi/wilddata/master/quests/cars.csv
```

Po wstępnej analizie okazało się jednak, że plik ten nie zawiera kolumny `price`, która jest niezbędna do wykonania zadania regresji. Zmienna ta powinna stanowić wartość docelową przewidywaną przez modele. Z tego powodu wykorzystano alternatywny zbiór danych z serwisu Kaggle: `aliiihussain/car-price-prediction`.

Dane pobrano w środowisku Google Colab przy użyciu biblioteki `kagglehub`:

```
1 import kagglehub
2
3 path = kagglehub.dataset_download("aliiihussain/car-price-
   prediction")
4 print("Path to dataset files:", path)
```

Następnie przygotowano funkcję wczytującą plik CSV do obiektu `DataFrame` biblioteki `pandas`:

```
1 import os
2 import pandas as pd
3
4 CAR_DATA_PATH = path
5
6 def load_car_data(car_path=CAR_DATA_PATH):
7     csv_path = os.path.join(car_path, "car_price_prediction_.csv")
8     return pd.read_csv(csv_path)
9
10 car_data = load_car_data()
```

3 Charakterystyka pierwszego zbioru danych

Pierwszy wykorzystany zbiór zawierał 2500 rekordów oraz 10 kolumn. Każdy rekord opisywał jeden samochód. W zbiorze występowały zarówno zmienne numeryczne, jak i kategoryczne.

3.1 Pierwsze pięć rekordów

Pierwsze pięć przykładów ze zbioru danych przedstawiono w tabeli 1.

Tabela 1: Pierwsze pięć rekordów pierwszego zbioru danych

Car ID	Brand	Year	Engine Size	Fuel Type	Transmission	Mileage	Condition	Price	Model
1	Tesla	2016	2.3	Petrol	Manual	114832	New	26613.92	Model X
2	BMW	2018	4.4	Electric	Manual	143190	Used	14679.61	5 Series
3	Audi	2013	4.5	Electric	Manual	181601	New	44402.61	A4
4	Tesla	2011	4.1	Diesel	Automatic	68682	New	86374.33	Model Y
5	Ford	2009	2.6	Diesel	Manual	223009	Like New	73577.10	Mustang

Kolumna `Price` została przyjęta jako zmienna docelowa. Kolumna `Car ID` pełniła jedynie funkcję identyfikatora rekordu, dlatego nie została wykorzystana jako cecha wejściowa.

3.2 Struktura danych

Na podstawie metody `info()` stwierdzono, że zbiór nie zawiera brakujących wartości. Każda kolumna posiadała 2500 niepustych obserwacji. Typy danych przedstawiały się następująco:

- 3 kolumny typu `int64`,
- 2 kolumny typu `float64`,
- 5 kolumn typu `object`.

Kolumny numeryczne:

- `Car ID`,
- `Year`,
- `Engine`,
- `Mileage`,
- `Price`.

Kolumny kategoryczne:

- `Brand`,
- `Fuel Type`,
- `Transmission`,
- `Condition`,
- `Model`.

3.3 Statystyki opisowe

Podstawowe statystyki opisowe dla cech numerycznych przedstawiono w tabeli 2.

Tabela 2: Statystyki opisowe pierwszego zbioru danych

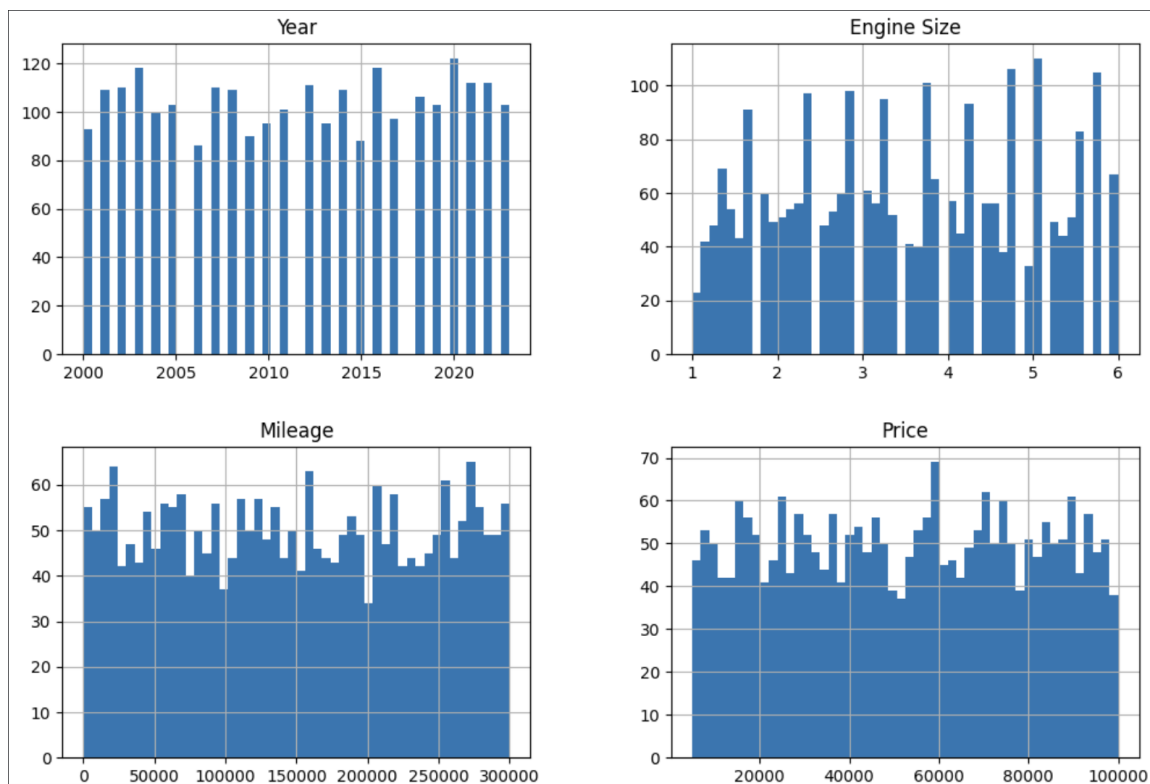
Statystyka	Car ID	Year	Engine Size	Mileage	Price
count	2500.00	2500.00	2500.00	2500.00	2500.00
mean	1250.50	2011.63	3.47	149749.84	52638.02
std	721.83	6.99	1.43	87919.95	27295.83
min	1.00	2000.00	1.00	15.00	5011.27
25%	625.75	2005.00	2.20	71831.50	28908.49
50%	1250.50	2012.00	3.40	149085.00	53485.24
75%	1875.25	2018.00	4.70	225990.50	75838.53
max	2500.00	2023.00	6.00	299967.00	99982.59

Średni rok produkcji samochodów wynosił około 2011.63, średnia pojemność silnika około 3.47l, średni przebieg około 149750 mil, a średnia cena około 52638 dolarów. Ceny samochodów mieściły się w zakresie od około 5011 do około 99983 dolarów.

4 Wstępna analiza i wizualizacja danych

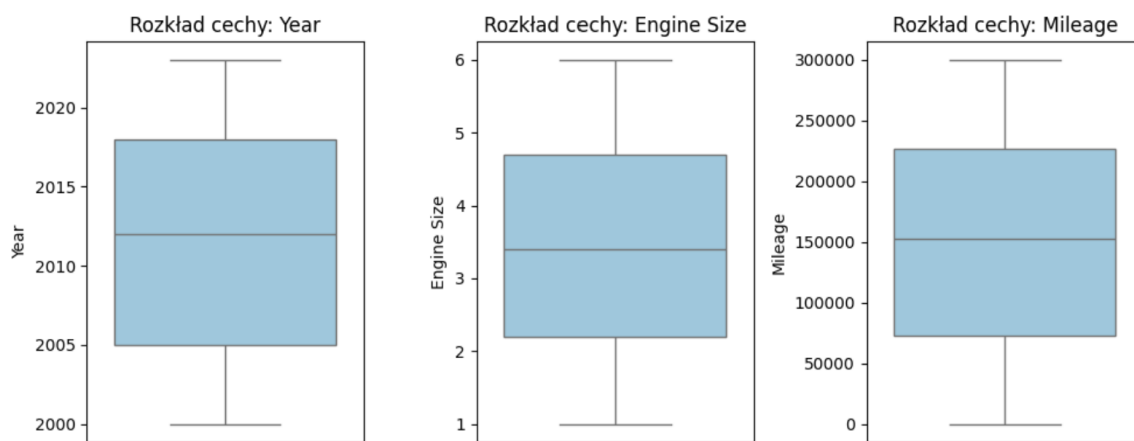
W celu lepszego poznania danych wykonano histogramy dla cech numerycznych. Pomińto kolumnę `Car ID`, ponieważ jest ona identyfikatorem i nie ma znaczenia predykcyjnego. Analizowane były między innymi rozkłady cech `Year`, `Engine Size`, `Mileage` oraz `Price`.

Na podstawie histogramów stwierdzono, że rozkłady cech numerycznych są stosunkowo równomierne. Nie zaobserwowano silnych rozkładów typu *tail-heavy* lub *head-heavy*. Oznacza to, że nie było wyraźnej potrzeby stosowania transformacji logarytmicznej cech.



Rysunek 1: Histogramy wybranych cech numerycznych pierwszego zbioru danych: roku produkcji, pojemności silnika, przebiegu oraz ceny.

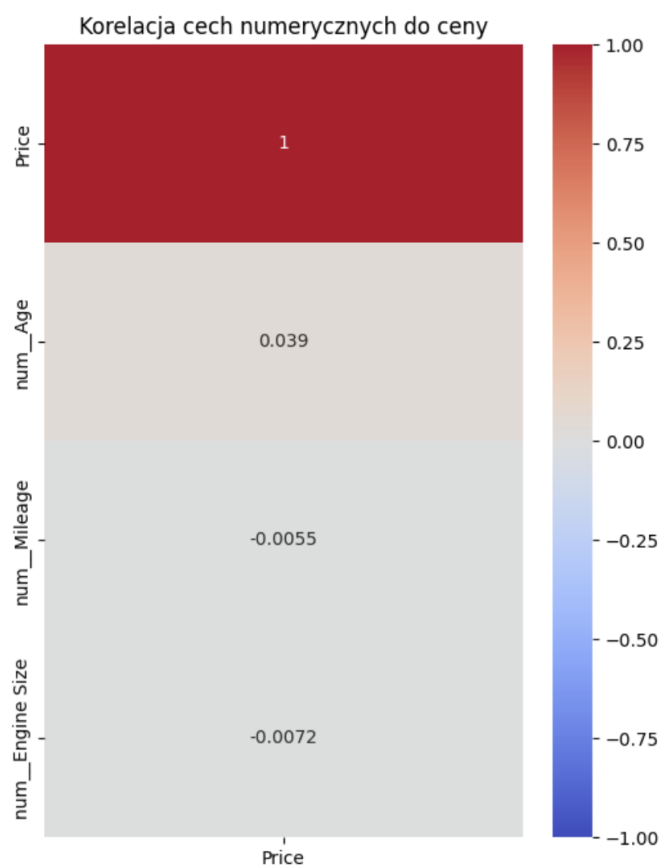
Dodatkowo wykonano wykresy pudełkowe dla cech Year, Engine Size i Mileage. Boxploty nie wykazały dużej liczby wartości odstających. Ten wynik wspiera wybór RMSE jako jednej z podstawowych metryk oceny, ponieważ RMSE jest wrażliwe na wartości odstające.



Rysunek 2: Wykresy pudełkowe cech numerycznych: rok produkcji, pojemność silnika oraz przebieg.

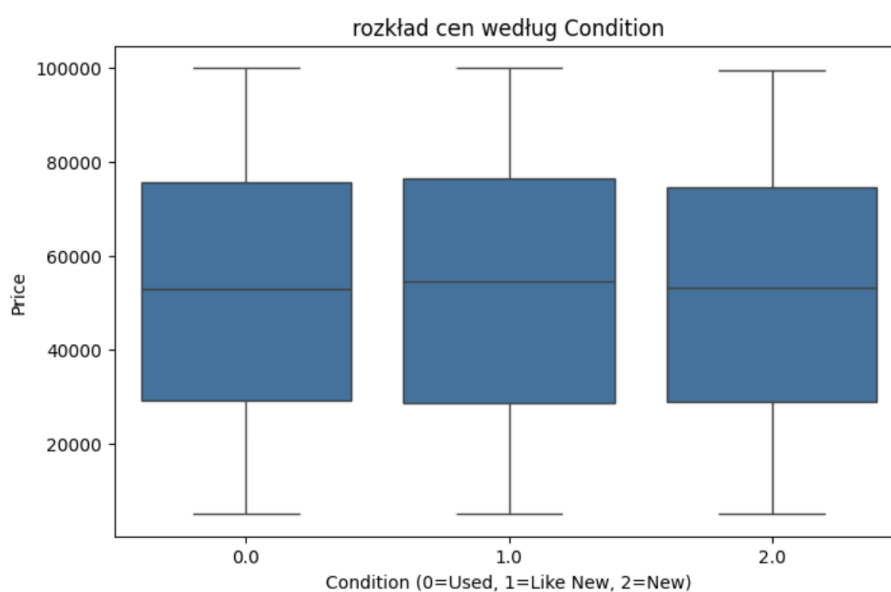
4.1 Korelacje z ceną

Po przetworzeniu danych zbadano korelację cech numerycznych z ceną. Otrzymane wartości były bardzo niskie:

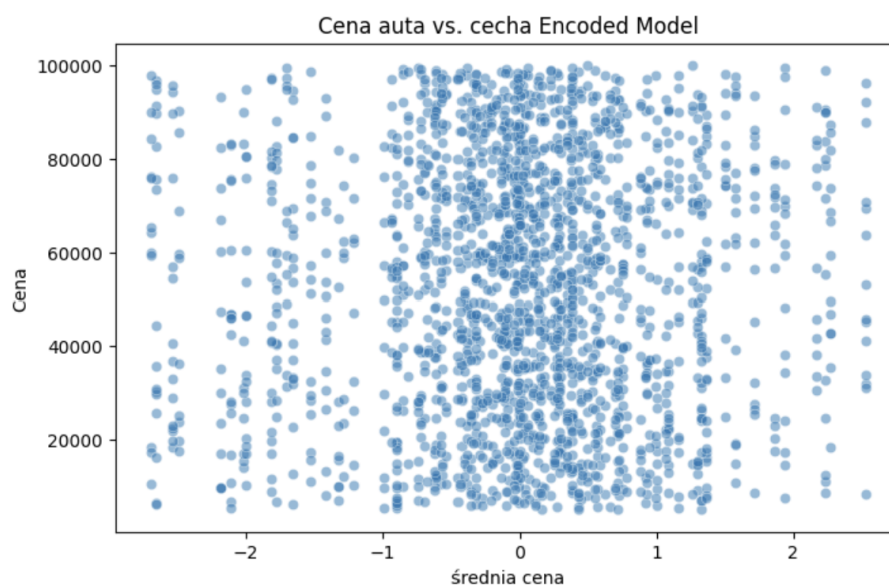


Rysunek 3: Korelacja przetworzonych cech numerycznych z ceną samochodu w pierwszym zbiorze danych.

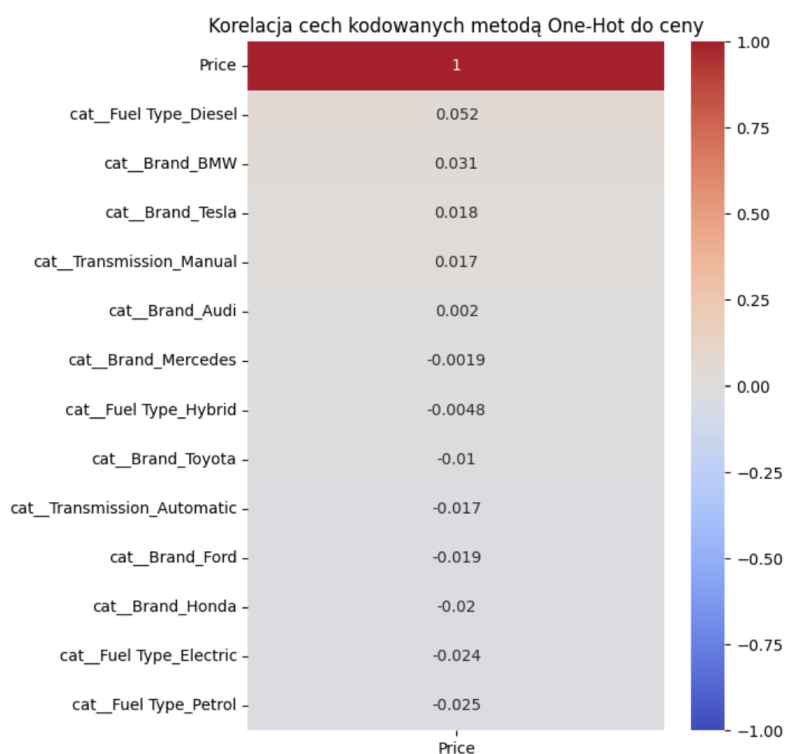
Oznacza to, że pojedyncze cechy numeryczne nie wykazują silnej liniowej zależności z ceną samochodu. Podobne obserwacje uzyskano dla cech kategorycznych zakodowanych metodą One-Hot. Wszystkie wartości korelacji z ceną były bardzo niskie, przykładowo Fuel Type_Diesel miała korelację około 0.052, a Brand_BMW około 0.031.



Rysunek 4: Rozkład cen samochodów względem zakodowanej cechy Condition.



Rysunek 5: Cena samochodu względem zakodowanej cechy Model1.



Rysunek 6: Korelacja cech kategorycznych zakodowanych metodą One-Hot z ceną samochodu.

Wyniki te sugerowały, że zależności w danych nie są dobrze opisywane przez proste zależności liniowe. Z tego względu uzasadnione było zastosowanie modeli nieliniowych, takich jak Random Forest, SVR oraz MLP.

5 Podział danych

Dane podzielono na zbiory treningowy, walidacyjny i testowy. Najpierw usunięto kolumny Car ID oraz Price. Kolumna Price została zapisana osobno jako zmienna docelowa.

```
1 X = car_data.drop(columns=["Car ID", "Price"])
2 y = car_data["Price"]
```

Następnie wykonano podział:

```
1 from sklearn.model_selection import train_test_split
2
3 X_train_full, X_temp, y_train_full, y_temp = train_test_split(
4     X, y, test_size=0.2, random_state=42
5 )
6
7 X_val, X_test, y_val, y_test = train_test_split(
8     X_temp, y_temp, test_size=0.5, random_state=42
9 )
```

Ostatecznie otrzymano podział przedstawiony w tabeli 3.

Tabela 3: Podział pierwszego zbioru danych

Zbiór	Liczba rekordów	Liczba cech
Treningowy	2000	8
Walidacyjny	250	8
Testowy	250	8

Zbiór treningowy służył do uczenia modeli, zbiór walidacyjny do porównywania wyników i doboru hiperparametrów, natomiast zbiór testowy powinien być wykorzystywany dopiero do końcowej oceny modelu.

6 Przygotowanie danych do uczenia maszynowego

Aby modele mogły korzystać z danych, konieczne było przekształcenie zmiennych kategorycznych do postaci numerycznej oraz przeskalowanie cech liczbowych.

W pierwszym kroku cecha Year została przekształcona do cechy Age. Wiek samochodu jest bardziej intuicyjny z punktu widzenia ceny pojazdu niż sam rok produkcji.

```
1 def calculate_age(X):
2     X_out = X.copy()
3     current_year = 2025
4     X_out['Age'] = current_year - X_out['Year']
5     return X_out.drop(columns=['Year'])
```

Dla cech numerycznych zastosowano pipeline składający się z przekształcenia Year na Age, uzupełniania braków medianą oraz standaryzacji:

```
1 num_pipeline = make_pipeline(
2     age_transformer,
3     SimpleImputer(strategy="median"),
```

```

4     StandardScaler()
5 )

```

Mimo że zbiór nie zawierał brakujących wartości, zastosowanie `SimpleImputer` zabezpiecza pipeline na wypadek pojawienia się braków w przyszłych danych.

Dla cechy `Condition` zastosowano `OrdinalEncoder`, ponieważ stan samochodu ma naturalny porządek:

```

1 condition_categories = ["Used", "Like New", "New"]

```

Oznacza to kodowanie: `Used` jako 0, `Like New` jako 1 oraz `New` jako 2. Dla zmiennych nominalnych `Transmission`, `Fuel Type` i `Brand` zastosowano `OneHotEncoder`. Dla cechy `Model` zastosowano `TargetEncoder`, ponieważ jest to cecha kategoriowa, której wartości mogą mieć związek ze średnią ceną samochodu.

Całość połączono za pomocą `ColumnTransformer`:

```

1 preprocessing_pipeline = ColumnTransformer(
2     transformers=[
3         ("num", num_pipeline, numerical_features),
4         ("ord", OrdinalEncoder(categories=condition_categories),
5           ordinal_features),
6         ("cat", OneHotEncoder(handle_unknown="ignore",
7           sparse_output=False), onehot_features),
8         ("target", target_pipeline, target_features)
9     ],
10    remainder="drop"
11 )

```

Pipeline został dopasowany wyłącznie na zbiorze treningowym:

```

1 X_train_prepared = preprocessing_pipeline.fit_transform(
2     X_train_full, y_train_full)
3 X_val_prepared = preprocessing_pipeline.transform(X_val)
4 X_test_prepared = preprocessing_pipeline.transform(X_test)

```

Po przetworzeniu danych otrzymano 18 cech wejściowych. Były to między innymi: `num__Engine Size`, `num__Mileage`, `num__Age`, `ord__Condition`, cechy zakodowane metodą One-Hot oraz `target__Model`.

7 Modele bazowe

Przed trenowaniem bardziej zaawansowanych modeli przygotowano modele bazowe. Ich zadaniem było wyznaczenie punktu odniesienia.

Pierwszym modelem był `DummyRegressor`, który przewidywał średnią cenę ze zbioru treningowego. Drugim modelem była regresja liniowa. Wyniki przedstawiono w tabeli 4.

Tabela 4: Wyniki modeli bazowych dla pierwszego zbioru danych

Model	RMSE na zbiorze walidacyjnym
DummyRegressor	27172.58
LinearRegression	27252.88

Regresja liniowa nie poprawiła wyniku względem modelu przewidującego średnią. Potwierdziło to wcześniejsze obserwacje z analizy korelacji: w danych nie występowały istotne zależności liniowe, które mogłyby zostać dobrze wykorzystane przez prosty model liniowy.

8 RandomForestRegressor

Pierwszym modelem opracowanym w projekcie był `RandomForestRegressor`. Las losowy składa się z wielu drzew decyzyjnych. Każde drzewo uczy się na losowej próbce danych i losowym podzbiorze cech. Predykcja końcowa w regresji jest średnią predykcji poszczególnych drzew.

Model bazowy Random Forest został uruchomiony z parametrem `n_estimators=100`. Wyniki przedstawiono w tabeli 5.

Tabela 5: Wyniki bazowego modelu Random Forest

Model	RMSE treningowy	RMSE walidacyjny
<code>RandomForestRegressor</code>	10524.82	28017.61

Błąd treningowy był dużo niższy niż walidacyjny, co wskazywało na overfitting. Model dopasował się do danych treningowych, ale słabo generalizował na dane walidacyjne.

Następnie wykonano optymalizację hiperparametrów przy użyciu `RandomizedSearchCV` oraz `GridSearchCV`. Testowano między innymi liczbę drzew, maksymalną głębokość drzew, minimalną liczbę próbek w liściu, liczbę cech branych pod uwagę przy podziale oraz użycie bootstrapu.

Najlepsze parametry znalezione przez `RandomizedSearchCV` i potwierdzone przez `GridSearchCV` były następujące:

```
n_estimators = 200
max_depth = 10
min_samples_leaf = 15
max_features = sqrt
bootstrap = True
```

Najlepszy wynik walidacji krzyżowej wyniósł 27165.66 RMSE. Po sprawdzeniu najlepszego modelu na zbiorze walidacyjnym otrzymano RMSE równe 27237.92. Wynik był bardzo zbliżony do modelu bazowego `DummyRegressor`, co sugeruje, że dostępne cechy pierwszego zbioru danych miały ograniczoną informacyjność.

9 SVR

Drugim modelem był **SVR**, czyli Support Vector Regression. SVR jest regresyjną odmianą metody wektorów nośnych. Model ten próbuje znaleźć funkcję, która przewiduje wartości z błędem mieszczącym się w określonym marginesie epsilon. Punkty znajdujące się wewnątrz tego marginesu nie są karane w funkcji straty.

Dla SVR optymalizowano następujące hiperparametry: `kernel`, `C`, `gamma` oraz `epsilon`. Najlepsze parametry znalezione przez `RandomizedSearchCV` były następujące:

```
kernel = poly
gamma = 0.01
epsilon = 5000
C = 100000
```

Wyniki modelu przedstawiono w tabeli 6.

Tabela 6: Wyniki modelu SVR dla pierwszego zbioru danych

Model	RMSE CV	RMSE walidacyjny
SVR po optymalizacji	27249.75	27181.38

Najlepszym jądrem okazało się jądro wielomianowe `poly`. Potwierdza to, że zależności w danych nie były dobrze opisywane przez model liniowy. Mimo tego wynik SVR nadal był bardzo zbliżony do wyniku modelu bazowego.

10 MLPRegressor

Trzecim modelem był `MLPRegressor`, czyli wielowarstwowa sieć neuronowa typu perceptron wielowarstwowy. Model ten składa się z warstwy wejściowej, jednej lub większej liczby warstw ukrytych oraz warstwy wyjściowej. Neurony w warstwach ukrytych przetwarzają dane z użyciem funkcji aktywacji, dzięki czemu sieć może uczyć się zależności nieliniowych.

W projekcie zastosowano `MLPRegressor` z parametrem `early_stopping=True`, co oznacza, że trening może zostać zatrzymany wcześniej, jeżeli wynik walidacyjny przestaje się poprawiać.

Optymalizowano następujące hiperparametry: liczbę warstw i neuronów, siłę regularyzacji L2, początkowe tempo uczenia oraz funkcję aktywacji. Najlepsze parametry były następujące:

```
hidden_layer_sizes = (50, 50)
activation = relu
alpha = 1.0
learning_rate_init = 0.001
```

Wyniki modelu przedstawiono w tabeli 7.

Tabela 7: Wyniki modelu MLPRegressor dla pierwszego zbioru danych

Model	RMSE CV	RMSE walidacyjny
MLPRegressor po optymalizacji	27449.14	27277.86

Najlepszą funkcją aktywacji w testowanym zakresie okazała się `relu`. Najlepsza architektura składała się z dwóch warstw ukrytych po 50 neuronów. Mimo zastosowania sieci neuronowej wynik pozostał podobny do innych modeli.

11 Porównanie wyników dla pierwszego zbioru danych

Zestawienie wyników walidacyjnych dla pierwszego zbioru danych przedstawia tabela 8.

Tabela 8: Porównanie wyników modeli dla pierwszego zbioru danych

Model	RMSE walidacyjny
DummyRegressor	27172.58
LinearRegression	27252.88
RandomForestRegressor	27237.92
SVR	27181.38
MLPRegressor	27277.86

Różnice pomiędzy modelami były niewielkie. Żaden z bardziej zaawansowanych modeli nie poprawił znacząco wyniku względem prostego modelu przewidującego średnią cenę. Oznacza to, że pierwszy zbiór danych prawdopodobnie nie zawierał wystarczająco silnego sygnału predykcyjnego.

Można stwierdzić, że problem nie wynikał wyłącznie z wyboru algorytmu, ale przede wszystkim z ograniczeń danych. Cechy takie jak marka, rok, pojemność silnika, przebieg, rodzaj paliwa, skrzynia biegów, stan i model nie pozwoliły w tym konkretnym zbiorze na dokładne przewidywanie ceny.

12 Analiza wyników innych notebooków Kaggle

W trakcie projektu porównano wyniki z innymi publicznymi notebookami Kaggle. Jeden z notebooków uzyskał bardzo niski błąd RMSE oraz bardzo wysokie R^2 dla modelu Random Forest. Po analizie kodu stwierdzono jednak, że wynik ten był spowodowany wyciekami danych.

Autor utworzył cechę:

```
Price_per_CC = Price / Engine Size
```

Cecha ta została obliczona bezpośrednio na podstawie zmiennej docelowej *Price*. W efekcie model otrzymał informację pozwalającą niemal odtworzyć cenę:

$$Price \approx Price_per_CC \times Engine\ Size \quad (2)$$

13 Drugi eksperyment na innym zbiorze danych

Ze względu na słabe wyniki modeli na pierwszym zbiorze danych zdecydowano się przeprowadzić dodatkowy eksperyment na innym zbiorze danych. Wykorzystano zbiór:

```
https://www.kaggle.com/datasets/taeefnajib/used-car-price-prediction-dataset/data
```

Celem drugiego eksperymentu było sprawdzenie, czy bardziej szczegółowy zbiór danych pozwoli uzyskać lepsze wyniki predykcji.

Drugi zbiór zawierał między innymi następujące cechy: `brand`, `model`, `milage`, `fuel_type`, `transmission`, `ext_col`, `int_col`, `accident`, `clean_title`, `price`, informacje o silniku oraz rok modelu.

W tym zbiorze wykonano dodatkowe czyszczenie danych i feature engineering. Kolumna `price` została oczyszczona ze znaków \$ i przecinków, a kolumna `milage` z tekstu `mi..` Z kolumny `engine` wyodrębniono moc `hp`, pojemność silnika `engine displacement` oraz informację, czy silnik jest typu V. Na podstawie roku modelu utworzono cechę `Vehicle_Age`, a dodatkowo obliczono `Mileage_per_Year`.

14 Charakterystyka zmiennych w drugim zbiorze danych

Drugi zbiór danych zawierał bardziej szczegółowe informacje o samochodach używanych niż pierwszy dataset. Dzięki temu możliwe było uwzględnienie większej liczby cech mających potencjalny wpływ na cenę pojazdu. Poniżej objaśniono znacznie każdej z kolumn w tym zbiorze.

- `brand` oraz `model` – marka oraz konkretny model pojazdu.
- `model_year` – rok produkcji lub rok modelowy pojazdu.
- `milage` – przebieg samochodu w milach.
- `fuel_type` – rodzaj paliwa lub napędu, na przykład benzyna, diesel, samochód elektryczny albo hybrydowy.
- `engine` – opis silnika. Kolumna ta zawierała informacje o parametrach jednostki napędowej, takich jak moc, pojemność oraz typ silnika. Na jej podstawie utworzono później dodatkowe cechy: `hp`, `engine displacement` oraz `is_v_engine`.
- `transmission` – typ skrzyni biegów, na przykład automatyczna, manualna lub inny wariant przekładni.
- `ext_col` oraz `int_col` – kolor zewnętrzny oraz kolor wnętrza pojazdu.
- `accident` – informacja o historii wypadków lub uszkodzeń pojazdu.
- `clean_title` – informacja o tym, czy samochód posiada czysty tytuł własności.
- `price` – cena samochodu.

15 Podział drugiego zbioru danych

Po oczyszczeniu i przygotowaniu drugiego zbioru danych wydzielono zmienną docelową oraz cechy wejściowe. Zmienną docelową była kolumna `price`, natomiast pozostałe kolumny zostały wykorzystane jako cechy opisujące samochody.

Dane zostały podzielone na trzy części: zbiór treningowy, walidacyjny oraz testowy. Najpierw wydzielono 20% danych jako zbiór tymczasowy, a następnie podzielono go po równo na zbiór walidacyjny i testowy. Ostatecznie uzyskano podział 80% / 10% / 10%.

- zbiór treningowy: 3207 obserwacji i 14 cech,
- zbiór walidacyjny: 401 obserwacji i 14 cech,
- zbiór testowy: 401 obserwacji i 14 cech.

Zbiór treningowy został wykorzystany do uczenia modeli. Zbiór walidacyjny służył do porównywania jakości modeli oraz wyboru najlepszych hiperparametrów. Zbiór testowy został użyty dopiero na końcu eksperymentu, aby ocenić skuteczność wytrenowanych modeli na danych, które nie były wykorzystywane podczas uczenia ani optymalizacji.

16 Preprocessing drugiego zbioru danych

Dla drugiego zbioru danych przygotowano osobny pipeline przetwarzający dane. Jego zadaniem było uzupełnienie brakujących wartości, zakodowanie zmiennych kategoriycznych oraz standaryzacja cech numerycznych.

Cechy numeryczne:

- milage,
- hp,
- engine displacement,
- Vehicle_Age,
- Mileage_per_Year.

Dla cech numerycznych zastosowano uzupełnianie brakujących wartości medianą oraz standaryzację za pomocą `StandardScaler`. Standaryzacja była potrzebna, ponieważ część modeli, szczególnie `SVR` oraz `MLPRegressor`, jest wrażliwa na skalę danych wejściowych.

Cechy o wysokiej kardynalności:

- brand,
- model,
- transmission,
- ext_col,
- int_col.

Dla tych cech zastosowano `TargetEncoder`, ponieważ zawierały wiele unikalnych wartości i mogły przechowywać istotną informację związaną z ceną samochodu. Przykładowo marka, model pojazdu lub typ skrzyni biegów mogą znacząco wpływać na wartość samochodu.

Cechy kodowane metodą One-Hot:

- fuel_type,
- accident,
- clean_title,
- is_v_engine.

Dla tych cech zastosowano uzupełnianie brakujących wartości najczęściej występującą wartością oraz kodowanie metodą `OneHotEncoder`. Metoda ta została użyta, ponieważ wymienione cechy mają charakter kategoriyczny i nie posiadają naturalnego porządku liczbowego.

17 Wyniki modeli na drugim zbiorze danych

W drugim eksperymencie modele optymalizowano przy użyciu metryki MAE, czyli średniego błędu bezwzględnego:

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad (3)$$

Następnie obliczono zarówno RMSE, jak i MAE dla zbioru walidacyjnego. Wyniki walidacyjne przedstawia tabela 9.

Tabela 9: Wyniki walidacyjne modeli dla drugiego zbioru danych

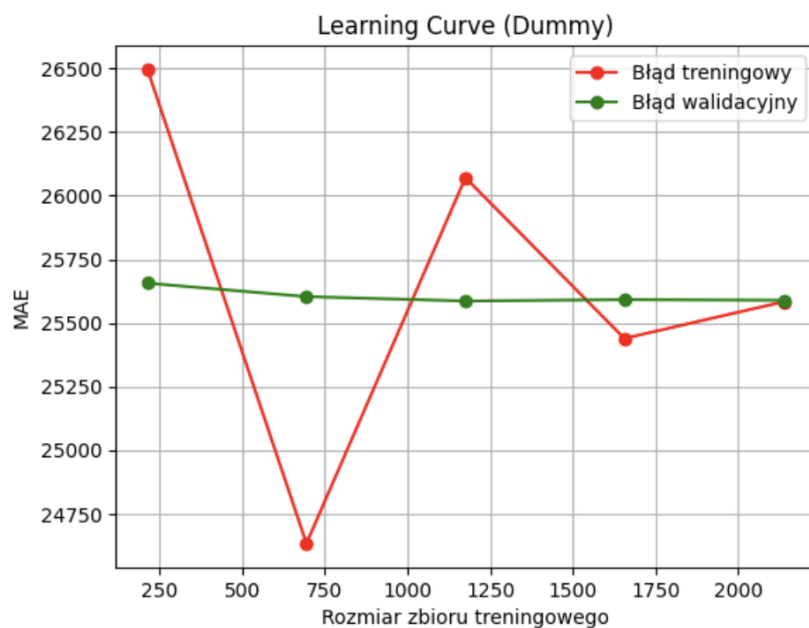
Model	RMSE walidacyjny	MAE walidacyjny
DummyRegressor	48952.46	25083.04
LinearRegression	29886.96	14817.35
RandomForestRegressor	21608.34	8303.17
SVR	27283.44	10228.56
MLPRegressor	26359.42	11119.21

Wyniki drugiego eksperymentu były znacznie lepsze niż w przypadku pierwszego zbioru. Szczególnie dobrze poradził sobie **RandomForestRegressor**, który uzyskał najniższy błąd zarówno pod względem RMSE, jak i MAE.

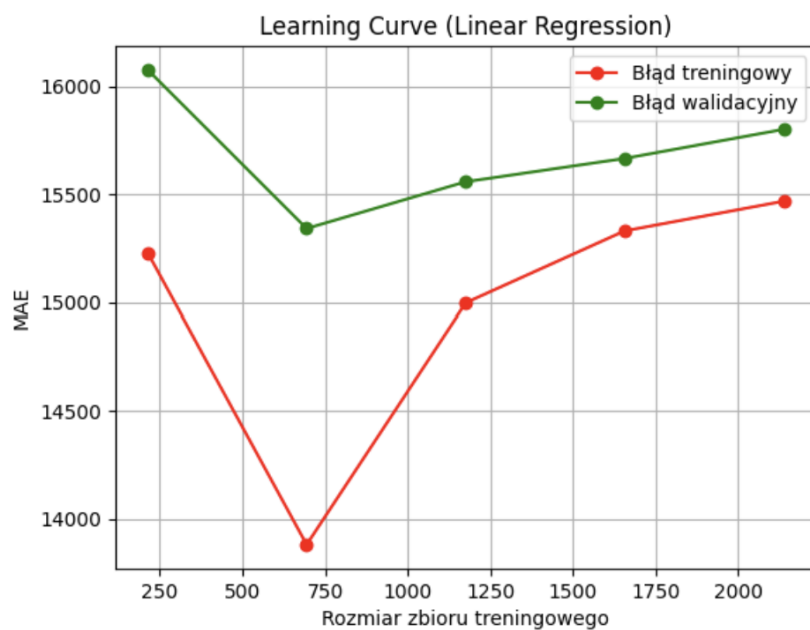
Oznacza to, że drugi zbiór danych zawierał więcej użytecznych informacji predykcyjnych. Cechy takie jak marka, model, przebieg, wiek pojazdu, moc i pojemność silnika lepiej wyjaśniały cenę samochodu.

18 Krzywe uczenia

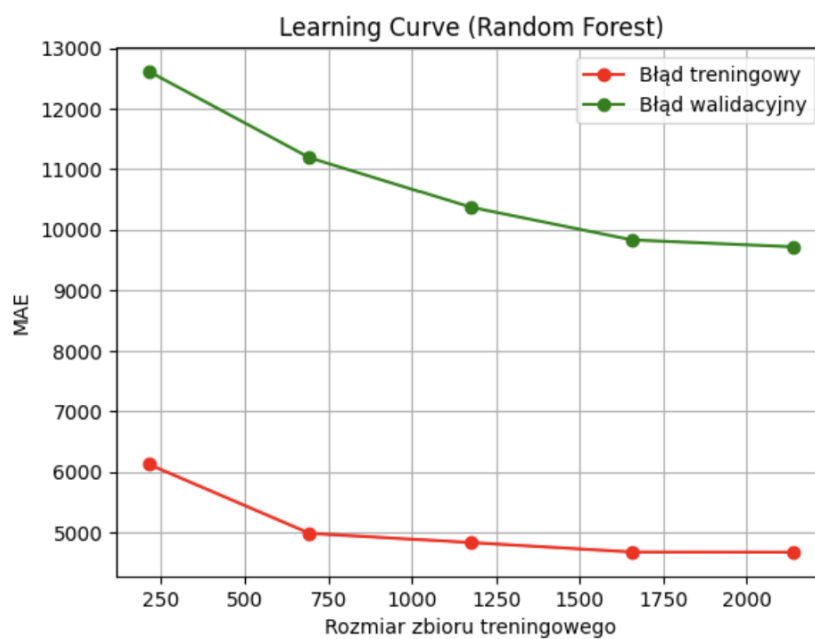
Dla modeli wytrenowanych na drugim zbiorze danych przygotowano krzywe uczenia. Krzywe te pokazują, jak zmienia się błąd treningowy i walidacyjny wraz ze wzrostem liczby przykładów treningowych.



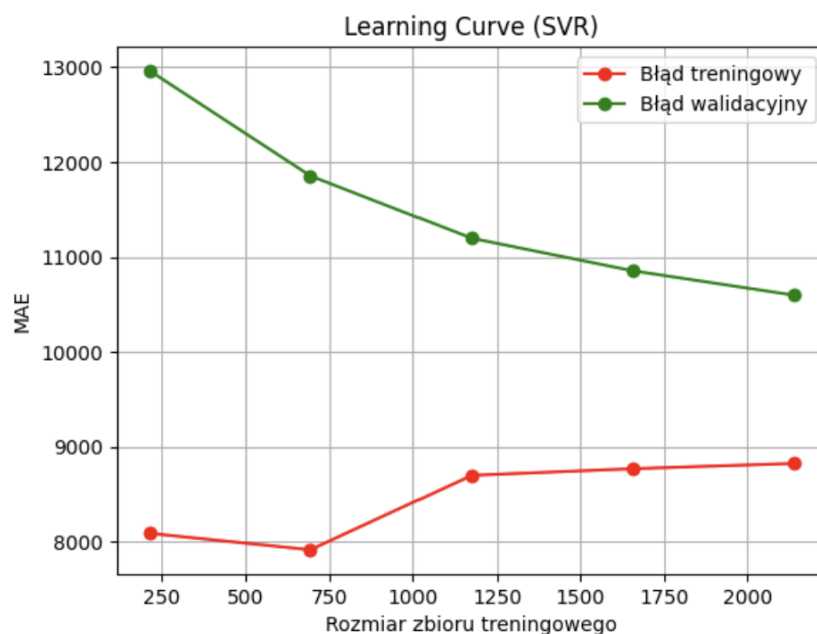
Rysunek 7: Krzywa uczenia dla modelu bazowego **DummyRegressor**.



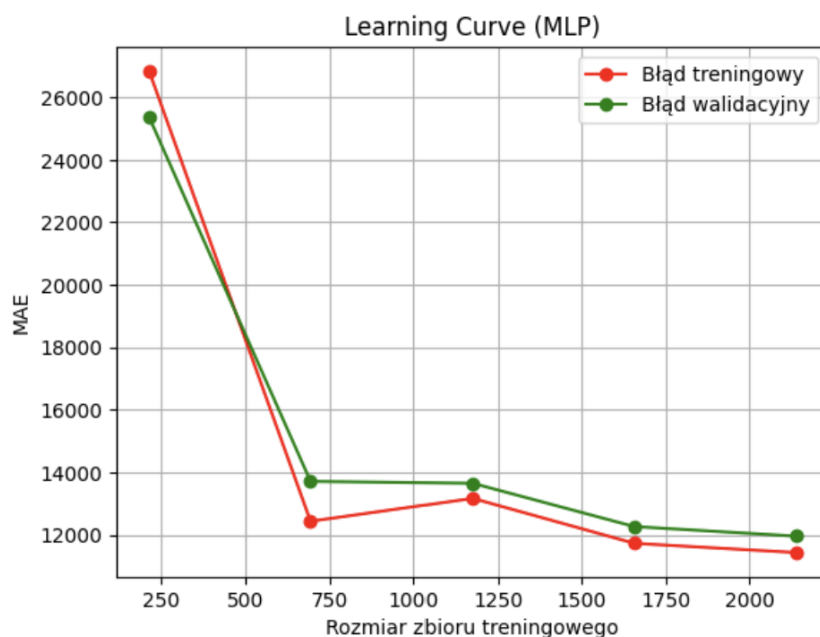
Rysunek 8: Krzywa uczenia dla modelu LinearRegression.



Rysunek 9: Krzywa uczenia dla modelu RandomForestRegressor.



Rysunek 10: Krzywa uczenia dla modelu SVR.



Rysunek 11: Krzywa uczenia dla modelu MLPRegressor.

Dla `DummyRegressor` błąd pozostawał wysoki, ponieważ model nie uczy się zależności pomiędzy cechami a ceną. Przewiduje jedynie wartość stałą.

Dla regresji liniowej wynik był wyraźnie lepszy niż dla modelu `Dummy`, ale nadal ograniczony przez liniowy charakter modelu.

Dla `RandomForestRegressor` uzyskano najniższy błąd walidacyjny. Widoczna była jednak różnica między błędem treningowym a walidacyjnym, co wskazuje na częściowy overfitting. Mimo tego model najlepiej generalizował spośród testowanych algorytmów.

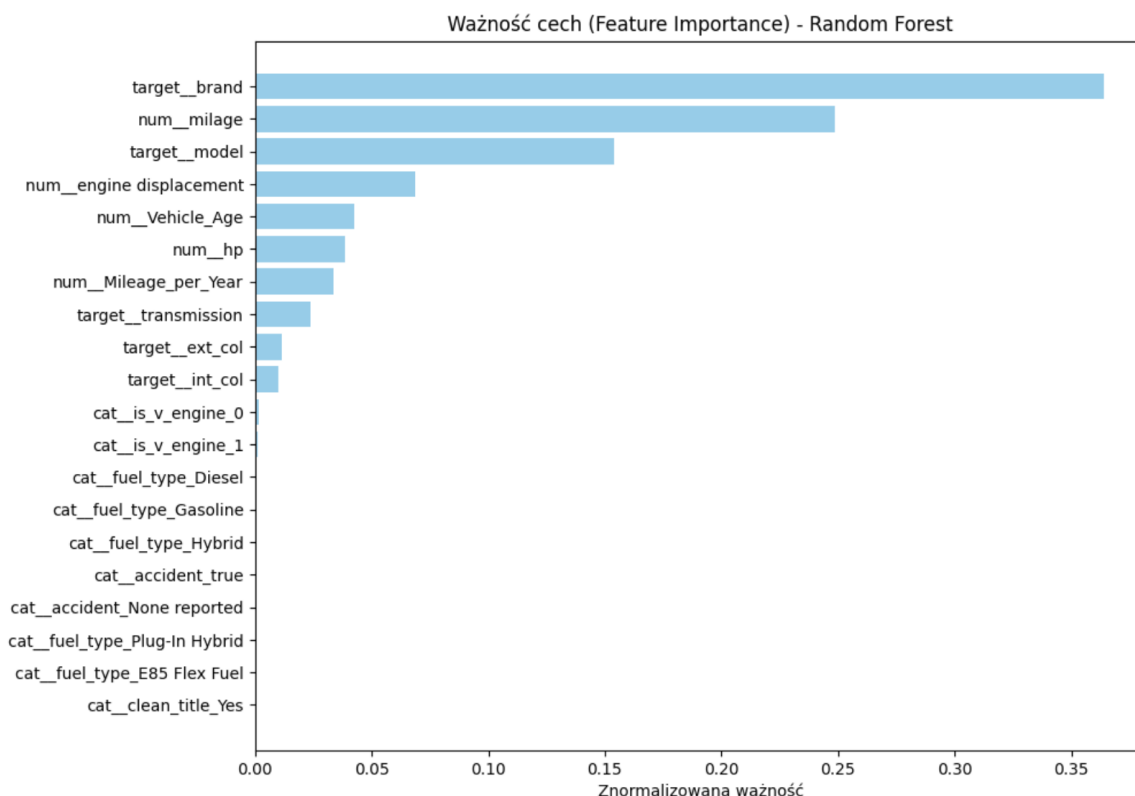
Dla SVR błąd walidacyjny spadał wraz ze wzrostem liczby danych treningowych, co oznacza, że model korzystał z większej ilości danych. Nie osiągnął jednak wyniku `Random`

Forest.

Dla `MLPRegressor` błąd był bardzo wysoki przy małej liczbie danych, ale wyraźnie spadał wraz ze wzrostem zbioru treningowego. Sieć neuronowa potrzebowała większej liczby przykładów, aby nauczyć się użytecznych zależności.

19 Ważność cech w modelu Random Forest

Dla najlepszego modelu, czyli Random Forest, przeprowadzono analizę ważności cech. Najważniejsze okazały się:



Rysunek 12: Ważność cech według modelu `RandomForestRegressor` dla drugiego zbioru danych.

- `target__brand`,
- `num__milage`,
- `target__model`,
- `num__engine displacement`,
- `num__Vehicle_Age`,
- `num__hp`,
- `num__Mileage_per_Year`.

Największy wpływ na przewidywaną cenę miały więc marka, przebieg, model, pojemność silnika, wiek pojazdu oraz moc. Cechy takie jak typ paliwa, informacja o wypadku, czysty tytuł czy typ silnika V miały znacznie mniejsze znaczenie.

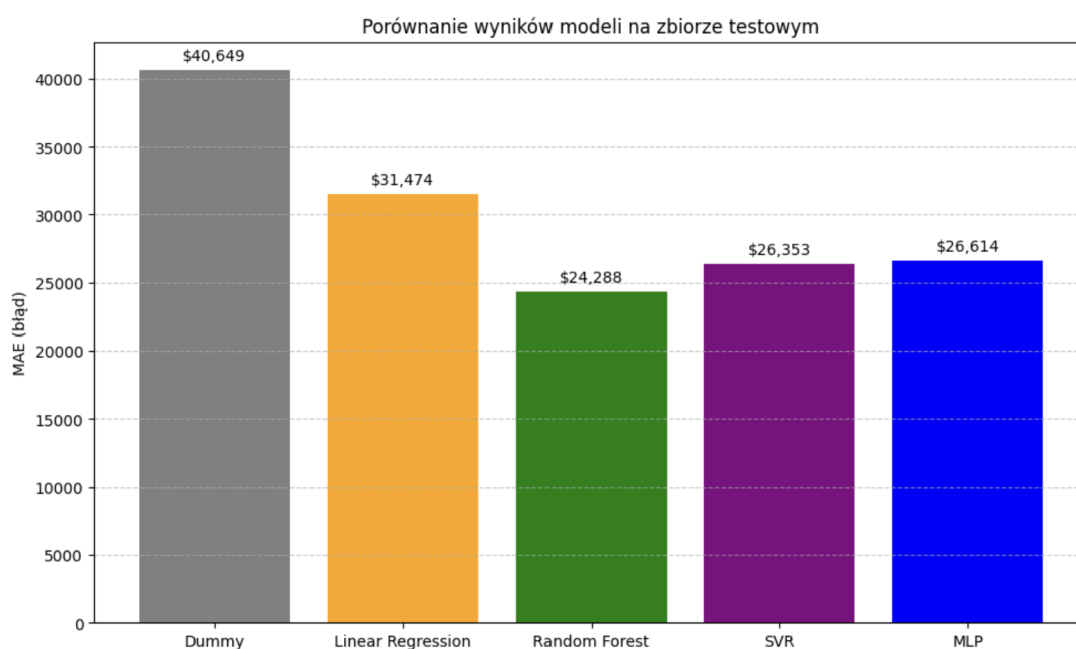
Wynik ten jest zgodny z intuicją, ponieważ cena samochodu używanego jest zwykle silnie zależna od marki, konkretnego modelu, przebiegu oraz wieku auta.

20 Ewaluacja końcowa na zbiorze testowym

Na końcu wykonano ewaluację modeli na zbiorze testowym drugiego datasetu. Zastosowano metrykę MAE. Wyniki testowe przedstawia tabela 10.

Tabela 10: Wyniki końcowe na zbiorze testowym drugiego datasetu

Model	MAE na zbiorze testowym
DummyRegressor	40649
LinearRegression	31474
RandomForestRegressor	24288
SVR	26353
MLPRegressor	26614



Rysunek 13: Porównanie modeli na zbiorze testowym według metryki MAE.

Najlepszy wynik na zbiorze testowym uzyskał `RandomForestRegressor`. Model ten osiągnął najniższy średni błąd bezwzględny, co oznacza, że przeciętnie mylił się mniej niż pozostałe algorytmy.

SVR oraz `MLPRegressor` również poprawiły wynik względem regresji liniowej i modelu Dummy, ale były słabsze od Random Forest.

21 Porównanie działania algorytmów

`LinearRegression` zakłada liniową zależność pomiędzy cechami wejściowymi a ceną. W pierwszym zbiorze nie poradziła sobie lepiej niż model bazowy, co wskazywało na brak silnych zależności liniowych.

`RandomForestRegressor` wykorzystuje wiele drzew decyzyjnych, które dzielą przestrzeń cech na mniejsze obszary. Model ten dobrze radzi sobie z zależnościami nieliniowymi i interakcjami między cechami. W drugim eksperymencie okazał się najlepszym modelem.

SVR próbuje dopasować funkcję regresji z uwzględnieniem marginesu epsilon. Dzięki zastosowaniu różnych jąder może modelować zależności nieliniowe. W pierwszym eksperymencie najlepszym jądrem było `poly`, natomiast w drugim testowano jądra `linear` i `rbf`.

`MLPRegressor` jest siecią neuronową, która uczy się zależności poprzez aktualizację wag między neuronami. Dzięki funkcjom aktywacji, takim jak `relu` lub `tanh`, może modelować zależności nieliniowe. W pierwszym eksperymencie najlepszą funkcją aktywacji było `relu`, a najlepszą architekturą były dwie warstwy ukryte po 50 neuronów.

22 Wnioski końcowe

W projekcie przeprowadzono pełny proces predykcji cen samochodów: od pobrania i analizy danych, przez preprocessing, aż po trenowanie, optymalizację i porównanie modeli regresyjnych.

Pierwszy zbiór danych okazał się mało informacyjny. Zaawansowane modele, takie jak Random Forest, SVR i `MLPRegressor`, nie uzyskały wyników istotnie lepszych od prostego modelu bazowego. Sugeruje to, że dostępne cechy nie zawierały wystarczającej ilości informacji do dokładnego przewidywania ceny samochodu.

Drugi eksperyment pokazał, że jakość i szczegółowość danych mają kluczowe znaczenie w projektach uczenia maszynowego. Po wykorzystaniu bogatszego zbioru danych wyniki modeli znacząco się poprawiły. Najlepszym modelem okazał się `RandomForestRegressor`, który uzyskał najniższe błędy walidacyjne oraz najlepszy wynik na zbiorze testowym.

Najlepsze wyniki hiperparametrów były następujące:

- dla Random Forest najlepsza okazała się większa liczba drzew i ograniczenie złożoności drzew przez parametry regularyzujące,
- dla SVR w pierwszym eksperymencie najlepsze było jądro wielomianowe `poly`,
- dla `MLPRegressor` najlepszą funkcją aktywacji było `relu`, a najlepszą architekturą były dwie warstwy ukryte po 50 neuronów.

Ostatecznie można stwierdzić, że sam wybór algorytmu nie wystarcza do uzyskania dobrego modelu predykcyjnego. Kluczowe znaczenie mają jakość danych, właściwe przygotowanie cech, poprawny podział na zbiory treningowe i testowe oraz rzetelna ewaluacja.